

Code Complexity Analysis

Table of Contents

1	Pseudocode	2
1.1	Function 1: rightRotation(node)	2
1.2	Function 2: leftRotation(node)	2
1.3	Function 3: twoRotations(node)	3
1.4	Function 4: balanceNode(node)	3
1.5	Function 5: balanceFactor(node)	3
1.6	Function 6: insert()	4
1.7	Function 7: findNodeByCoordinates()	4
1.8	Function 8: parseEUStations()	5
2	Time Complexity Analysis	6
2.1	Function: rightRotation	6
2.2	Function: leftRotation	6
2.3	Function: twoRotations	6
2.4	Function: balanceNode	7
2.5	Function: balanceFactor	7
2.6	Function: insert	8
2.7	Function: findNodeByCoordinates	8
2.8	Function: parseEUStations	8

1 Pseudocode

1.1 Function 1: rightRotation(node)

Location: /Tree/Avl.java

```
1 FUNCTION rightRotation(node):
2     leftChild = node.left
3     node.left = leftChild.right
4     leftChild.right = node
5
6     nodeHeight = 1 + MAX(
7         IF node.left      NULL THEN node.left.element.height ELSE -1,
8         IF node.right     NULL THEN node.right.element.height ELSE -1)
9
10    node.element.height = nodeHeight
11
12    leftHeight = 1 + MAX(
13        IF leftChild.left  NULL THEN leftChild.left.element.height ELSE -1,
14        IF leftChild.right  NULL THEN leftChild.right.element.height ELSE
15    -1)
16
17    leftChild.element.height = leftHeight
18
19    RETURN leftChild
20 END FUNCTION
```

1.2 Function 2: leftRotation(node)

Location: /Tree/Avl.java

```
1 FUNCTION leftRotation(node):
2     rightChild = node.right
3     node.right = rightChild.left
4     rightChild.left = node
5
6     nodeHeight = 1 + MAX(
7         IF node.left      NULL THEN node.left.element.height ELSE -1,
8         IF node.right     NULL THEN node.right.element.height ELSE -1)
9
10    node.element.height = nodeHeight
11
12    rightHeight = 1 + MAX(
13        IF rightChild.left  NULL THEN rightChild.left.element.height ELSE
14    -1,
15        IF rightChild.right  NULL THEN rightChild.right.element.height ELSE
16    -1)
17
18    rightChild.element.height = rightHeight
19
20    RETURN rightChild
21 END FUNCTION
```

1.3 Function 3: twoRotations(node)

Location: /Tree/Avl.java

```
1 FUNCTION twoRotations(node):
2     IF balanceFactor(node) > 0:
3         node.right = rightRotation(node.right)
4         node = leftRotation(node)
5     ELSE:
6         node.left = leftRotation(node.left)
7         node = rightRotation(node)
8     RETURN node
9 END FUNCTION
```

1.4 Function 4: balanceNode(node)

Location: /Tree/Avl.java

```
1 FUNCTION balanceNode(node):
2     balance = balanceFactor(node)
3
4     IF balance > 1: // Right-heavy
5         IF balanceFactor(node.right) < 0:
6             node = twoRotations(node) // Right-Left case
7         ELSE:
8             node = leftRotation(node) // Right-Right case
9
10    ELSE IF balance < -1: // Left-heavy
11        IF balanceFactor(node.left) > 0:
12            node = twoRotations(node) // Left-Right case
13        ELSE:
14            node = rightRotation(node) // Left-Left case
15
16    RETURN node
17 END FUNCTION
```

1.5 Function 5: balanceFactor(node)

Location: /Tree/Avl.java

```
1 FUNCTION balanceFactor(node):
2     rightHeight = IF node.right NULL THEN node.right.element.height ELSE -1
3     leftHeight = IF node.left NULL THEN node.left.element.height ELSE -1
4
5     RETURN rightHeight - leftHeight
6 END FUNCTION
```

Note: Heights are retrieved in $O(1)$ due to caching in each NodeData.

1.6 Function 6: insert()

Location: /Tree/Avl.java

```
1 FUNCTION insert(element):
2     root = insertHelper(element, root)
3 END FUNCTION
4
5 FUNCTION insertHelper(element, node):
6     IF node IS NULL:
7         RETURN new Node(element, NULL, NULL)
8
9     ELSE IF element < node.element:
10        node.left = insertHelper(element, node.left)
11    ELSE:
12        node.right = insertHelper(element, node.right)
13
14    leftHeight = IF node.left      NULL THEN node.left.element.height ELSE -1
15    rightHeight = IF node.right    NULL THEN node.right.element.height ELSE -1
16
17    node.element.height = 1 + MAX(leftHeight, rightHeight)
18
19    node = balanceNode(node)
20
21    RETURN node
22 END FUNCTION
```

1.7 Function 7: findNodeByCoordinates()

Location: /Tree/Avl.java

```
1 FUNCTION findNodeByCoordinates(coord):
2     RETURN findNodeByCoordinatesHelper(coord, root)
3 END FUNCTION
4
5 FUNCTION findNodeByCoordinatesHelper(coord, node):
6     IF node IS NULL:
7         RETURN NULL
8
9     data = node.getElement()
10    cmpLat = compare(coord.latitude, data.coordinate.latitude)
11    IF cmpLat = 0:
12        IF cmpLat < 0:
13            RETURN findNodeByCoordinatesHelper(coord, node.left)
14        ELSE:
15            RETURN findNodeByCoordinatesHelper(coord, node.right)
16
17    cmpLon = compare(coord.longitude, data.coordinate.longitude)
18    IF cmpLon = 0:
19        IF cmpLon < 0:
20            RETURN findNodeByCoordinatesHelper(coord, node.left)
21        ELSE:
22            RETURN findNodeByCoordinatesHelper(coord, node.right)
23
24    RETURN data
25 END FUNCTION
```

1.8 Function 8: parseEUStations()

Location: Parser/Parser64k.java

```
1 FUNCTION parseEUStations():
2     avl = new AVL()
3     inputStream = openResource("train_stations_europe.csv")
4     IF inputStream IS NULL:
5         THROW error "File not found"
6
7     reader = BufferedReader(InputStreamReader(inputStream))
8     header = reader.readLine() // skip header
9
10    FOR each line IN reader:
11        fields = split(line, ",")
12
13        IF length(fields) < 10:
14            CONTINUE // skip incomplete rows
15
16        tzGroup = TimeZoneGroup.fromString(fields[1])
17        IF tzGroup IS NULL:
18            CONTINUE
19
20        name = fields[2]
21        countryCode = fields[3]
22
23        IF name is empty OR countryCode is empty:
24            CONTINUE
25
26        location = new GeographicalLocation()
27        IF NOT parseCheckLatLong(fields[4], fields[5], location):
28            CONTINUE
29
30        isCity = (fields[6] == "True")
31        isMain = (fields[7] == "True")
32        isAirport = (fields[8] == "True")
33
34        station = new Station(location, tzGroup, ..., country, name, ...,
35        isCity, isAirport, isMain)
36
37        node = avl.findNodeByCoordinates(location)
38        IF node IS NOT NULL:
39            node.addStation(station) // add, keeping name order
40        ELSE:
41            avl.insert(new NodeData(location, station))
42
43    RETURN avl
44 END FUNCTION
```

2 Time Complexity Analysis

2.1 Function: rightRotation

Operation	Description	Best Case	Average Case	Worst Case	Notes
Get left child	Access node.left	$O(1)$	$O(1)$	$O(1)$	Pointer access
Reattach left's right	Set node.left = leftChild.right	$O(1)$	$O(1)$	$O(1)$	Pointer assignment
Attach node as rightChild	Set leftChild.right = node	$O(1)$	$O(1)$	$O(1)$	Pointer assignment
Compute nodeHeight	Height update via MAX of children	$O(1)$	$O(1)$	$O(1)$	Heights cached
Set node.element.height	Update height in node	$O(1)$	$O(1)$	$O(1)$	Integer assignment
Compute leftHeight	Height update for new root	$O(1)$	$O(1)$	$O(1)$	Heights cached
Set leftChild.height	Update height in new root	$O(1)$	$O(1)$	$O(1)$	Integer assignment
Return new root	Return leftChild	$O(1)$	$O(1)$	$O(1)$	Value return
TOTAL		$O(1)$	$O(1)$	$O(1)$	Constant time operation

2.2 Function: leftRotation

Operation	Description	Best Case	Average Case	Worst Case	Notes
Get right child	Access node.right	$O(1)$	$O(1)$	$O(1)$	Pointer access
Reattach right's left	Set node.right = rightChild.left	$O(1)$	$O(1)$	$O(1)$	Pointer assignment
Attach node as leftChild	Set rightChild.left = node	$O(1)$	$O(1)$	$O(1)$	Pointer assignment
Compute nodeHeight	Height update via MAX of children	$O(1)$	$O(1)$	$O(1)$	Heights cached
Set node.element.height	Update height in node	$O(1)$	$O(1)$	$O(1)$	Integer assignment
Compute rightHeight	Height update for new root	$O(1)$	$O(1)$	$O(1)$	Heights cached
Set rightChild.height	Update height in new root	$O(1)$	$O(1)$	$O(1)$	Integer assignment
Return new root	Return rightChild	$O(1)$	$O(1)$	$O(1)$	Value return
TOTAL		$O(1)$	$O(1)$	$O(1)$	Constant time operation

2.3 Function: twoRotations

Operation	Description	Best Case	Average Case	Worst Case	Notes
Compute balance factor	Call balanceFactor(node)	$O(1)$	$O(1)$	$O(1)$	
Right-Left case:	Right rotation on right child	$O(1)$	$O(1)$	$O(1)$	
	Left rotation on node	$O(1)$	$O(1)$	$O(1)$	
Left-Right case:	Left rotation on left child	$O(1)$	$O(1)$	$O(1)$	
	Right rotation on node	$O(1)$	$O(1)$	$O(1)$	
TOTAL		$O(1)$	$O(1)$	$O(1)$	Double rotation constant time

2.4 Function: balanceNode

Operation	Description	Best Case	Average Case	Worst Case	Notes
Compute balance factor	Calculate balanceFactor(node)	$O(1)$	$O(1)$	$O(1)$	Height cached
Check tree balance	Compare balance to 1 or -1	$O(1)$	$O(1)$	$O(1)$	Integer check
Branch to case	Select rotation type	$O(1)$	$O(1)$	$O(1)$	Conditional logic
Perform rotation	Call rotation functions	$O(1)$	$O(1)$	$O(1)$	Each rotation $O(1)$
Return subtree root	Return balanced node	$O(1)$	$O(1)$	$O(1)$	Pointer return
TOTAL		$O(1)$	$O(1)$	$O(1)$	Constant time rebalancing

2.5 Function: balanceFactor

Operation	Description	Best Case	Average Case	Worst Case	Notes
Access right height	Get right child height or -1	$O(1)$	$O(1)$	$O(1)$	Heights cached
Access left height	Get left child height or -1	$O(1)$	$O(1)$	$O(1)$	Heights cached
Subtract heights	Compute difference (right - left)	$O(1)$	$O(1)$	$O(1)$	Integer operation
TOTAL		$O(1)$	$O(1)$	$O(1)$	Single node computation

2.6 Function: insert

Operation	Description	Best Case	Average Case	Worst Case	Notes
Insert traversal	Recursively descend tree	$O(\log n)$	$O(\log n)$	$O(n)$	Balanced vs skewed
Compare elements	Determine insertion side	$O(1)$	$O(1)$	$O(1)$	Per comparison
Create new node	Leaf node creation	$O(1)$	$O(1)$	$O(1)$	Memory allocation
Update heights	Compute from children's heights	$O(1)$	$O(1)$	$O(1)$	Cached heights
Balance operations	Call balanceNode per ancestor	$O(1)$	$O(1)$	$O(1)$	Each rebalance $O(1)$
Pointer assignments	Link children/parents	$O(1)$	$O(1)$	$O(1)$	Reference updates
TOTAL (per insert)		$O(\log n)$	$O(\log n)$	$O(n)$	AVL maintains balance

2.7 Function: findNodeByCoordinates

Operation	Description	Best Case	Average Case	Worst Case	Notes
Compare latitude	Coordinate comparison	$O(1)$	$O(1)$	$O(1)$	Numeric comparison
Traverse children	Recursive calls	$O(1)$ per call	$O(1)$ per call	$O(1)$ per call	Single path
Compare longitude	Secondary comparison	$O(1)$	$O(1)$	$O(1)$	If latitudes equal
Tree traversal	Search until match/leaf	$O(\log n)$	$O(\log n)$	$O(n)$	Tree height dependent
Return result	Node data or NULL	$O(1)$	$O(1)$	$O(1)$	Constant return
TOTAL		$O(\log n)$	$O(\log n)$	$O(n)$	Logarithmic search

2.8 Function: parseEUStations

Operation	Description	Best Case	Average Case	Worst Case	Notes
Let: N = CSV lines, n = distinct coordinates, k = stations per coordinate					
Open resource	Initialize stream/reader	$O(1)$	$O(1)$	$O(1)$	One-time setup
Read & parse line	Process each CSV row	$O(1)$	$O(1)$	$O(1)$	Per line operation
Field validation	Check data integrity	$O(1)$	$O(1)$	$O(1)$	Constant checks
Coordinate parsing	Parse lat/long values	$O(1)$	$O(1)$	$O(1)$	Numeric conversion

Tree search	findNodeByCoordinates call	$O(\log n)$	$O(\log n)$	$O(n)$	Coordinate lookup
Station addition	Add to existing node	$O(1)$ - $O(k)$	$O(1)$	$O(k)$	Small k in practice
Tree insertion	AVL insert operation	$O(\log n)$	$O(\log n)$	$O(n)$	New coordinate
Process all data	Loop over N lines	$O(N)$	$O(N)$	$O(N)$	Linear in input size
TOTAL		$O(N \log n)$	$O(N \log n)$	$O(N n)$	Efficient bulk loading